Real-Time Data Pipeline Architecture with Apache Kafka and Spark Structured Streaming: A Market-Data Case Study

Group 6

Arya Zarei
Rawad Alharastani
Mark Samwaiel

Project category: In-Depth Analysis of Data Management Systems, with a case study that reproduces and extends published experiments.

1 Background and Significance

Financial firms process market data as a continuous stream of trades, and they need summaries and alerts within seconds of each trade. A common design pairs a durable, partitioned log, which stores incoming events, with a stream processor that reads the log and computes results over time windows.

Apache Kafka [1, 2] is the log. Each topic is split into partitions. A partition is an append-only file on a server (a broker), written and read sequentially in batches. Kreps et al. showed that this design gives much higher throughput than traditional message queues. Kafka keeps events in order only within a partition, so sending all trades for one stock symbol to the same partition keeps that symbol's trades in order. Each partition is copied to several brokers: one leader handles reads and writes, and followers that are fully caught up are called in-sync replicas. Whether a confirmed write survives a broker crash depends on three settings: the number of copies (the replication factor), how many copies the producer waits for before treating a write as done (the `acks` setting), and the minimum number of in-sync replicas required to accept a write. Recent versions manage cluster metadata with a built-in consensus protocol (KRaft) instead of ZooKeeper.

Spark Structured Streaming [3] is the processor. It treats a stream as a table that keeps growing and runs ordinary SQL-style queries over it, usually in small batches (micro-batches). This approach descends from Spark's earlier discretized streams [4]. It supports aggregation over time windows and keeps intermediate results (state) between batches. A watermark tells Spark how long to wait for late events before finalizing a window, an idea from the Dataflow model [5]. Spark records which Kafka offsets each batch read, along with its state, in a checkpoint directory. Kafka can re-deliver data from any offset, so after a crash Spark re-runs exactly the batches that had not finished. If the output is written so that repeating a write has no further effect, the final result is as if each event was processed exactly once. Spark also has an experimental continuous mode with lower latency, but it only supports simple per-record operations and can deliver an event more than once.

Significance. These papers claim high throughput from batched, sequential disk access, durability from replication, and exactly-once results from replay plus checkpoints. Each depends on configuration and on how the two systems are connected. We test them on a financial workload, focusing (unlike the Kafka-only course topic) on how Kafka and Spark work together.

2 Goals and Objectives

We will answer three research questions. (1) How do the number of partitions, how often Spark starts a batch, and the processing mode trade latency against throughput? (2) Which Kafka settings prevent lost or duplicated data when a broker crashes, and how long does Spark take to recover after its own crash? (3) How much does exactly-once output cost compared with at-least-once output?

1. Architectural analysis. For *Kafka*, we will explain brokers, partitions, log storage and retention, replication and in-sync replicas, consumer groups, and the three delivery guarantees (at-most-once, at-least-once and exactly-once). For *Spark*, we will explain micro-batch versus continuous execution, triggers (how often a batch starts), time windows and watermarks, the state store, checkpointing, and how Spark's query optimizer turns a query into an incremental plan. For the *integration*, we will explain how Kafka partitions map to Spark tasks, why Spark tracks offsets in its own checkpoint rather than through Kafka consumer groups, and which output designs give exactly-once results. We will also describe Kafka's security features (encryption, authentication and access control lists) without measuring them.

2. Case study pipeline. We will build a market-data pipeline that starts with one `docker compose up` command. A producer sends trades to a three-broker Kafka cluster, using the stock symbol as the message key. A Spark job computes one-minute price bars for each symbol (the open, high, low and close prices, and the volume-weighted average price) and writes them to PostgreSQL. In the same write step, it flags any bar whose price change is more than three standard deviations from that symbol's average over the previous 30 bars, and records it in an alerts table. A Grafana dashboard reads PostgreSQL and shows the bars and alerts live.

3. Reproduction and extension. On a laptop, we will reproduce two published results, comparing the shape of each curve rather than absolute numbers: (a) Kafka producer throughput without and with batching (Figure 4 of Kreps et al. [1]), using Kafka's built-in performance test tool; and (b) latency versus input rate for a simple pass-through job from Kafka to Kafka in continuous mode, against the maximum throughput of micro-batch mode (Figure 7 of Armbrust et al. [3], which also used a 4-core machine). We will extend these papers by measuring what neither one evaluated: whether the pipeline's output stays correct when a broker or the Spark job crashes, and what exactly-once output costs.

4. Reproducibility. A public repository will hold the code, a Docker Compose file with pinned versions, a README and benchmark scripts. Crashes are scripted, so reruns give the same correctness results.

3 Methodology

Data. Benchmarks use a synthetic trade generator with a fixed random seed. It produces random-walk prices for 100 symbols and injects large price jumps at known times. Each trade has a unique ID and a timestamp. Because the generator is repeatable, we can compute the correct bars and alerts separately and check the pipeline's output for lost or duplicated results. For the demo, a second producer streams live trades from Coinbase's public WebSocket feed, and can replay a saved copy if the connection drops.

Measurement. All containers run on one machine and share one clock. For the pass-through job, latency is the time between a trade's creation timestamp and the timestamp Kafka assigns when the output is written. For price bars, we measure the time from when a bar can be finalized (window end plus watermark) to when its row appears in PostgreSQL. Sustained throughput is the highest input rate at which Spark keeps up with Kafka over a five-minute run, meaning the gap

Table 1: Planned experiments.

Experiment	What we change	What we measure
1. Performance	Input rate, number of partitions, how often Spark starts a batch; micro-batch vs. continuous mode; producer batching (both reproductions)	Sustained throughput, latency
2. Broker crash	Force-kill the leader broker mid-run; producer waits for one copy vs. all in-sync copies (3 copies, at least 2 in sync)	Time writes are stalled, lost or duplicated trades
3. Spark crash	Force-kill Spark mid-run; restart from checkpoint	Recovery time, correctness
4. Exactly-once cost	Plain insert vs. insert-or-update on symbol and minute, killing Spark mid-write	Duplicate rows, cost in throughput and latency

between the newest offset and the offset Spark has processed does not grow. Each setting runs three times, and we report the median, 95th and 99th percentile latency.

4 Resources and Tools

Apache Kafka 4.4 (3 brokers); Apache Spark 4.2 Structured Streaming, written in Python (PySpark); PostgreSQL 18 for results; Grafana 13 for the dashboard; Python for the producers and analysis; Docker Compose on one 16 GB machine; and GitHub.

5 Challenges, Timeline and Work Division

Anticipated challenges. Everything shares one machine, so we compare trends, not absolute numbers. Continuous mode cannot run the bar job, so we compare modes on the pass-through job.

Milestones. Nov 3–14: Kafka cluster, generator and Spark job. Nov 15–21: PostgreSQL, dashboard and measurement scripts. Nov 22–30: experiments. Then: report, slides, and a recorded 15-minute talk and demo, with every member speaking.

Work division. *Kafka (Member 1):* cluster, generator and Coinbase producer; runs Experiment 2, the partition sweep and the batching reproduction. *Spark (Member 2):* Spark job and checkpointing; runs Experiments 3 and 4. *Output and measurement (Member 3):* PostgreSQL, dashboard and measurement scripts; runs the input-rate, batch-interval and processing-mode sweeps, and maintains the README and Compose file. Everyone shares the analysis, report and talk.

References

[1] J. Kreps, N. Narkhede, and J. Rao. Kafka: A distributed messaging system for log processing. In *NetDB*, 2011.

[2] G. Wang et al. Building a replicated logging system with Apache Kafka. *PVLDB*, 8(12):1654–1655, 2015.

[3] M. Armbrust et al. Structured Streaming: A declarative API for real-time applications in Apache Spark. In *SIGMOD*, pages 601–613, 2018.

[4] M. Zaharia et al. Discretized streams: Fault-tolerant streaming computation at scale. In *SOSP*, pages 423–438, 2013.

[5] T. Akidau et al. The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. *PVLDB*, 8(12):1792–1803, 2015.